RTA Fines & Salik Data Engine

Integration Proposal for Pilot GPS Platform

Prepared by TSC (Sky Electronics), Pilot partner #207, UAE. October 2026.

We have built a working UAE fines-and-tolls data engine (RTA traffic fines + Salik tolls) that is already live for a car-rental fleet on your platform, delivered today as a Pilot Store extension. This document proposes native integration paths so the data can appear inside Pilot itself — on the vehicle card and in reports — without a browser extension.

1. What the engine does today

Detail
Dubai letter-code plates via the comgw.rta.ae gateway (AES-encrypted); Sharjah/Abu Dhabi/Ajman/UAQ/RAK/Fujairah via UTS plate co
Pulls trips from two Salik portal accounts (Take Auto + KVADRAT plates) hourly from a UAE-side machine, pushes a JSON snapshot to
Maps Pilot vehicles (plate + agent ID) to RTA plate format (e.g. DXB K 62968, SHJ 3 18894) and Salik plates. Manual linking form includ
Per-vehicle trip/parking segments for any period via Pilot's tracks_get_history; click a fine/toll and the car's position at event time opens
Tier-1 support chat (DeepSeek) with email reports, one-time email verification, WhatsApp/Telegram escalation to the operator.

2. Why we propose native integration

Today the data lives in a Store extension tab. The operator wants the same numbers on the main page: when a car is clicked, the vehicle card should show its RTA fines and Salik spend alongside speed, status and sensors — the same way your sensor rows are shown. That is only possible if the data enters Pilot as native vehicle-linked values.

We analysed the current Pilot frontend (bundle): the vehicle popup is built by getMarkerContent2() from the marker data object, and sensor values are stored per-vehicle. There is no public API to push external values into a vehicle's sensor set from a third party. So the data cannot be “sent as sensor data” by us alone — it requires a small integration on your side.

3. Proposed integration options

Option A — External data API (minimal Pilot change, recommended)

Pilot exposes a small authenticated endpoint (e.g. POST /backend/ax/external_data.php) that accepts per-vehicle key/value pairs: plate/agent_id, values like rta_fines_count, rta_fines_amount_aed, salik_30d_trips, salik_30d_amount_aed. These values appear in the vehicle popup (one small addition to getMarkerContent2 or a dedicated “external data” row section) and optionally in reports.

Work on Pilot side	Estimated effort
New endpoint external_data.php (auth + upsert per agent_id)	1-2 dev days
Popup: render external values in the vehicle card	1 dev day
Optional: include values in fleet reports / CSV export	1-2 dev days

Our side: we push a fresh snapshot every hour (the Salik cadence) and immediately after each RTA refresh. We already run the puller infrastructure and a server (pilot-ai.zamtek.ae) that can post the

payloads.

Option B — Native engine embedding (deep integration)

Pilot embeds the data sources natively: you consume the RTA gateway and the Salik portals directly on your backend, with our know-how, plate-decode logic (emirate → UTS code mapping) and portal automation scripts handed over. The whole pipeline — lookup, caching, matching, reporting — becomes a first-class Pilot feature for all UAE fleets.

Work on Pilot side	Estimated effort
RTA gateway client (AES envelope, session, plate decode)	3-5 dev days
Salik portal automation (login, multi-account, geo requirement)	5-10 dev days
UI: fines/tolls tab + vehicle-card rows + reports	5-7 dev days

4. Technical handover material (what we can provide)

	Detail
	FastAPI server (Python) on pilot-ai.zamtek.ae with endpoints /salik, /rta-fines, /ask, /report, /send, /tts. Full source available.
	comgw.rta.ae API flow, AES key handling, plate-format decode per emirate (DXB letter codes; UTS numeric codes for other emirates
	Playwright script for customers.salik.ae multi-account login and trip scraping, hourly cron, JSON snapshot format. Must run from UAE
	Plate normalisation rules, vehicle → plate → Salik account mapping, linking data model.
	Current snapshot: 82 matched vehicles, 30-day toll history, format documented.

5. Sample payloads

Salik snapshot (excerpt)

{"matched": 82, "notFound": 67, "totalTolls": 11140.5, "days": 30, "data": {"AA35668 Duster": {"salikPlate": "35668:AA", "tripCount": 165, "totalTolls": 722.4, "tolls": [{"date": "2026-10-01 08:14", "amount": 4.0, "gate": "Al Barsha"}, …]}}}

RTA fines record

{"plate": "DXB K 62968", "fines": [{"ticketDate": "2026-09-12", "description": "Exceeding speed limit", "amount": 600}], "totalAmount": 600, "totalBlackPoints": 0}

6. What we ask from Pilot

#	Request
1	Confirm which option (A or B) Pilot prefers; we can provide a sandbox account for testing.
2	For Option A: define the endpoint contract (auth method, upsert semantics, value naming) together.
3	For Option B: a kickoff call with your backend team; we hand over source, scripts and live credentials handling.
4	Keep the current Store extension published until the native path is live, so the fleet keeps working.

7. Contact

TSC / Sky Electronics — Pilot partner #207, UAE. Contact: muhammed.charyyev@gmail.com.
Backend: https://pilot-ai.zamtek.ae (health, data and API docs available on request).